Software Book Recommender

A Hybrid Clustering & Similarity-based Recommendation Browser

Domain: Software Development Books | Tech Stack: Python • Scikit-Learn • Streamlit

Presented By: Abanoub Barayo

The Challenge of Discovery

Technical books suffer from unique selection challenges, making automated recommendation essential.

🔍 Skill-Level Mismatch

Generic algorithms group simple tutorials and advanced systems architecture books together.

💬 Review Sparsity & Bias

Ratings for technical materials are often highly clustered or sparse, skewing simple average calculations.

🏷️ Vague Metadata

Broad tags like 'Computer Science' hide specific paths (e.g. OOP patterns vs. Systems scripts).

Sourcing & Parallel Enrichment

We bypassed API rate limits and web scraping blocks using a hybrid data extraction framework.

1. Goodreads Web Scraping (500 Books)

Parsed user-curated book list pages for initial metadata (titles, authors, ratings, and count of ratings) using BeautifulSoup.

2. Open Library API (500 Books)

Queried specific programming subjects like 'algorithms' and 'web-development' to pull page counts and categories.

3. Multi-threaded Execution (10x Speedup)

Utilized python's ThreadPoolExecutor to request book descriptions and page counts concurrently, reducing API latency from 15 minutes to under 90 seconds.

Exploratory Data Analysis

Analyzing data shapes and core distributions helped identify anomalies and structure the clustering boundaries.

⭐ Average Rating Distribution

Strong left-skewed density centered around a median of 3.95 out of 5. Simple random recommendations would over-index on positive values, requiring similarity features to target content specific features.

📄 Page Count Distributions

Highly varied page range (50 to 1500+). We handled entries with missing pages by imputing the median value (250 pages) to protect the model clusters from severe outliers.

📊 The Long Tail of Popularity

Ratings count distributions follow a strict power law. A tiny subset of classic programming guides have 10,000+ ratings, whereas niche topics have under 50. Handled via log scaling.

Feature Engineering Pipeline

How we transform text descriptions and book stats into high-dimensional coordinate arrays.

1. TF-IDF Text Vectorization

Combined genre strings and description texts into a unified document. Weighted genres 2x to prioritize core domains. TF-IDF Vectorizer extracted top 1,500 features (with English stop words filter).

2. PCA Dimensionality Reduction

Compressed the sparse 1,500-dimension text matrices using PCA down to 15 dense principal components to eliminate background noise and accelerate execution speed.

3. StandardScaler & Stack fusion

Scaled structural features (ratings and page counts) via StandardScaler. Applied 0.5 scale weighting factor, and stacked horizontally with PCA dense text features.

The Machine Learning Core

A layered strategy that segments general topics first and then matches individual items.

K-Means Clustering (10 Clusters)

Partitions the library space into 10 tracks (e.g. Software Architecture, Front-end UI, Python scripting, Java web frameworks) serving as logical starting paths.

Cosine Similarity Matrix

Measures directional correlation between the 17D coordinates, locating neighbors with minimal angular variance.

Coherence Logic Restraints

Recommendations filter matches first inside the input's assigned K-Means cluster to protect context integrity, switching to general cosine matching only as a fallback.

DEMO

Key Outcomes

✓ Constructed a reliable database of 1,000+ programming books by solving scraping limitations and parallel API connections.

✓ Combined textual analysis (descriptions) and numeric profiles (ratings/lengths) into 17D semantic representations.

✓ Delivered a performant, custom UI that filters, clusters, and displays recommendations instantly without reloading page resources.

Thank You

Github Repository: https://github.com/abanoubdev/book-recommendation-browser

Streamlit : https://sotfware-ironhack-book-recommendation-browser.streamlit.app